

SyzSpec: Specification Generation for Linux Kernel Fuzzing via Under-Constrained Symbolic Execution

Presenter: Haonan Li¹

Author: Yu Hao¹, Juefei Pu¹, Xingyu Li¹, Zhiyun Qian¹, Ardalan Amiri Sani²

¹University of California, Riverside, ²University of California, Irvine



Kernel Fuzzing, Syzkaller & Syscall Descriptions

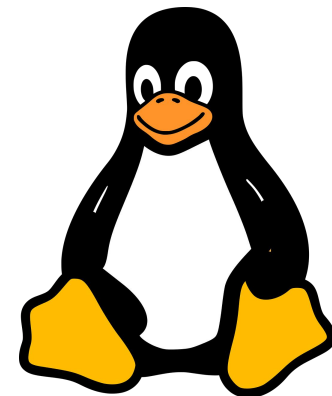
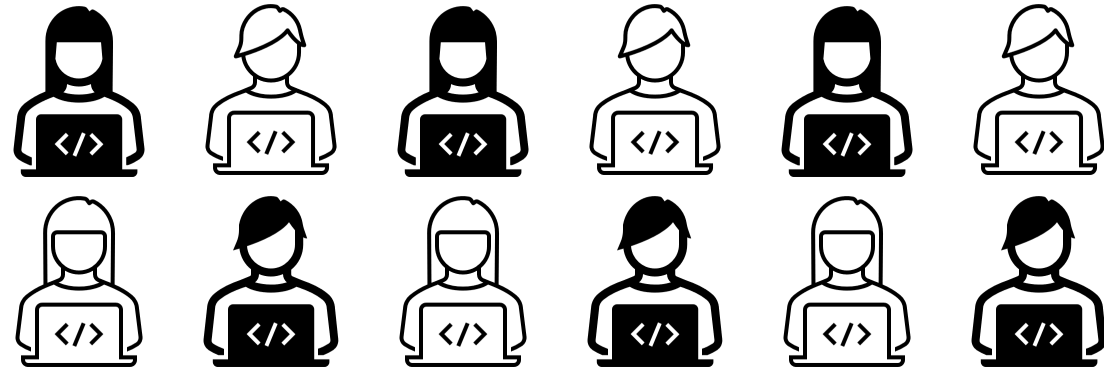
Syzkaller
(state-of-the-art kernel fuzzer)



Syscall Descriptions

**time-consuming
error-prone**

Fuzzing:
syscalls



Linux kernel and Device
Drivers

Found more than
6k bugs!



Syscall Descriptions

§ Syscall interface

§ Dependency

§ User inputs

```
1. resource fd_sndseq[fd]
2. openat$sndseq(fd const, file ptr[in, string["/dev/snd/seq"]], flags flags[open_flags]) fd_sndseq
3. ioctl$CMD(fd fd_sndseq,
    cmd const[CMD], arg ptr[in, snd_seq_queue_info])
4. snd_seq_queue_info {
5.   queue          int32
6.   owner          int32
7.   locked         int32:1
8.   name           string[snd_seq_queue_name, 64]
9.   flags          int32
10.  reserved       array[const[0, int8], 60]
11. }
12. read$sndseq(fd fd_sndseq,
    data ptr[out, array[snd_seq_event]], len bytesize[data])
13. write$sndseq(fd fd_sndseq,
    data ptr[in, array[snd_seq_event]], len bytesize[data])
```

Syscall Descriptions

§ Syscall interface

§ Dependency

§ User inputs

```
1. resource fd_sndseq[fd]
2. openat$sndseq(fd const, file ptr[in, string["/dev/snd/seq"]], flags flags[open_flags]) fd_sndseq
3. ioctl$CMD(fd fd_sndseq, cmd const[CMD], arg ptr[in, snd_seq_queue_info])
4. snd_seq_queue_info {
5.   queue          int32
6.   owner          int32
7.   locked         int32:1
8.   name           string[snd_seq_queue_name, 64]
9.   flags          int32
10.  reserved       array[const[0, int8], 60]
11. }
12. read$sndseq(fd fd_sndseq, data ptr[out, array[snd_seq_event]], len bytesize[data])
13. write$sndseq(fd fd_sndseq, data ptr[in, array[snd_seq_event]], len bytesize[data])
```

Syscall Descriptions

§ Syscall interface

§ Dependency

§ User inputs

```
1. resource fd_sndseq[fd]
2. openat$sndseq(fd const, file ptr[in, string["/dev/snd/seq"]], flags flags[open_flags]) fd_sndseq
3. ioctl$CMD(fd fd_sndseq,
  cmd const[CMD], arg ptr[in, snd_seq_queue_info])
4. snd_seq_queue_info {
5.   queue          int32
6.   owner          int32
7.   locked         int32:1
8.   name           string[snd_seq_queue_name, 64]
9.   flags          int32
10.  reserved       array[const[0, int8], 60]
11. }
12. read$sndseq(fd fd_sndseq,
  data ptr[out, array[snd_seq_event]] len bytesize[data])
13. write$sndseq(fd fd_sndseq,
  data ptr[in, array[snd_seq_event]] len bytesize[data])
```

Syscall Descriptions

§ Syscall interface

§ Dependency

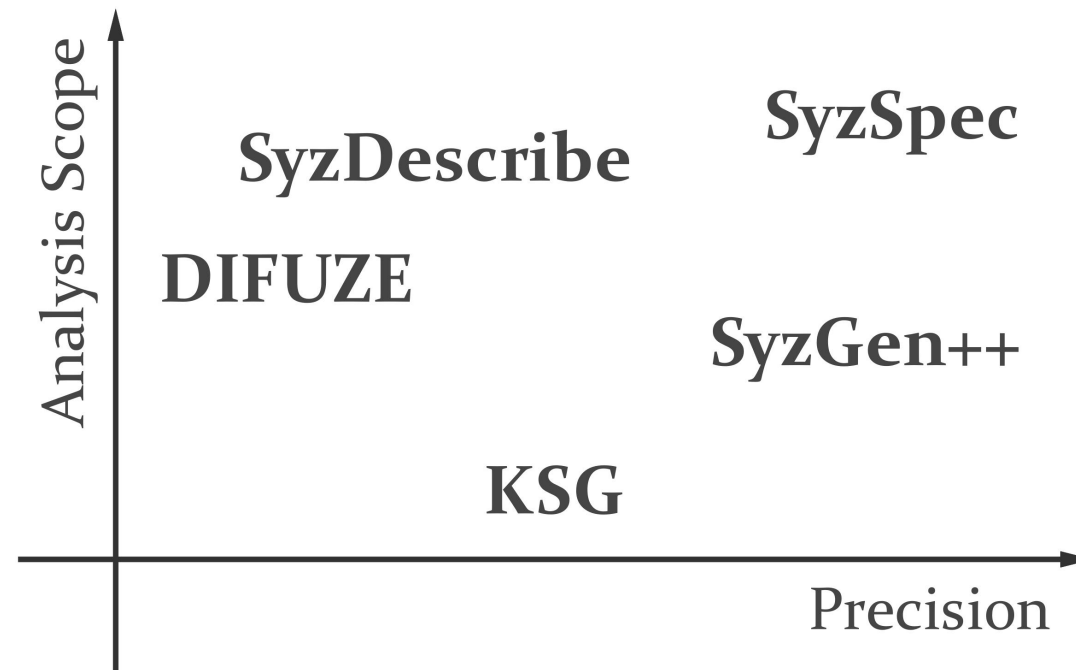
§ User inputs

```
1. io_uring_register$IORING_REGISTER_BUFFERS2(fd fd_io_uring,  
2.      opcode const[IORING_REGISTER_BUFFERS2],  
3.      arg ptr[in, io_uring_rsrc_register], size bytesize[arg])  
4.  
5. io_uring_rsrc_register {  
6.     nr      len[data, int32]  
7.     flags   flags[io_uring_rsrc_flags, int32]  
8.     resv2   const[0, int64]  
9.     data    ptr64[in, array[iovec_out]]  
10.    tags    ptr64[in, array[int64]]  
11. }
```

Syscall Description Generation

Name	Technique		Target
DIFUZE (2017 CCS)	static analysis	inter-procedural (w/o indirect call)	ioctl
SyzDescribe (2023 S&P)	static analysis	inter-procedural	ioctl
KSG (2022 USENIX ATC)	symbolic execution	intra-procedural	ioctl, read, write
SyzGen++ (2024 S&P)	symbolic execution based on Angr with memory snapshot	inter-procedural (w/o indirect call/ symbolic function pointer)	ioctl, read, write
SyzSpec	under-constrained symbolic execution	inter-procedural	user inputs for all syscalls

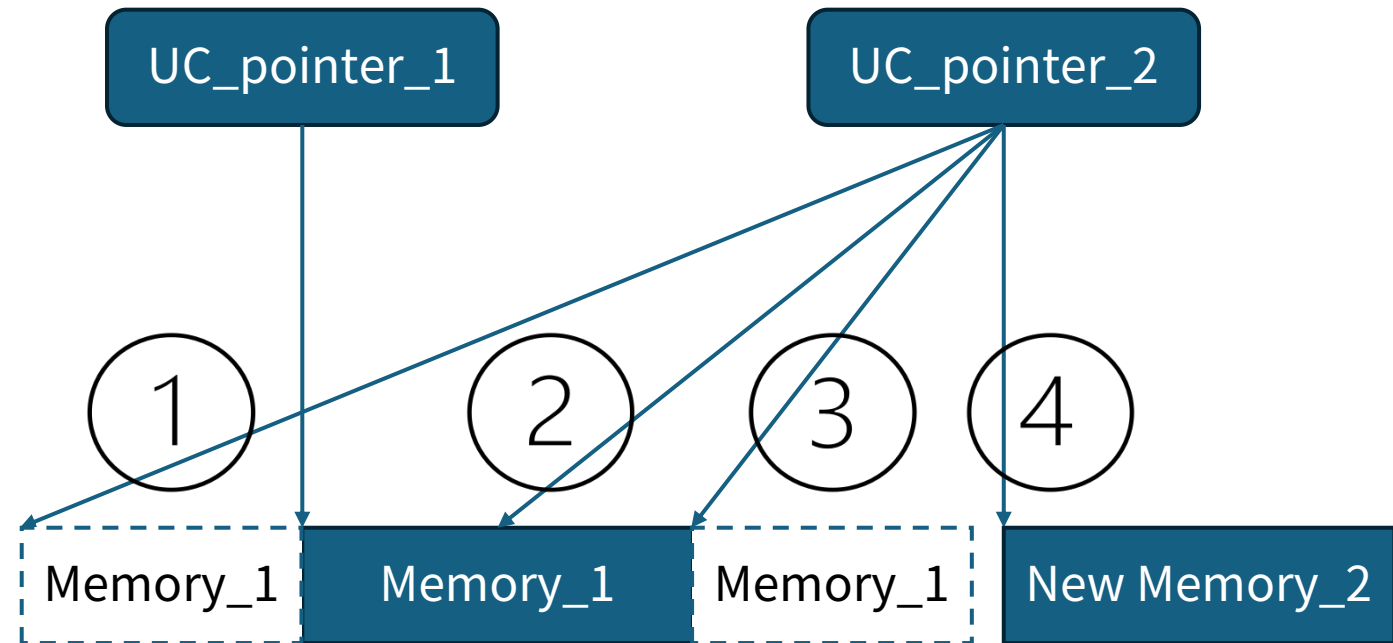
Conceptual Comparison of Different Tools



Symbolic Pointer Reasoning in Under-Constrained Symbolic Execution

- Under-Constrained Symbolic Variable (including pointer)
 - A **new** memory object?
 - Or any of the **existing** objects?

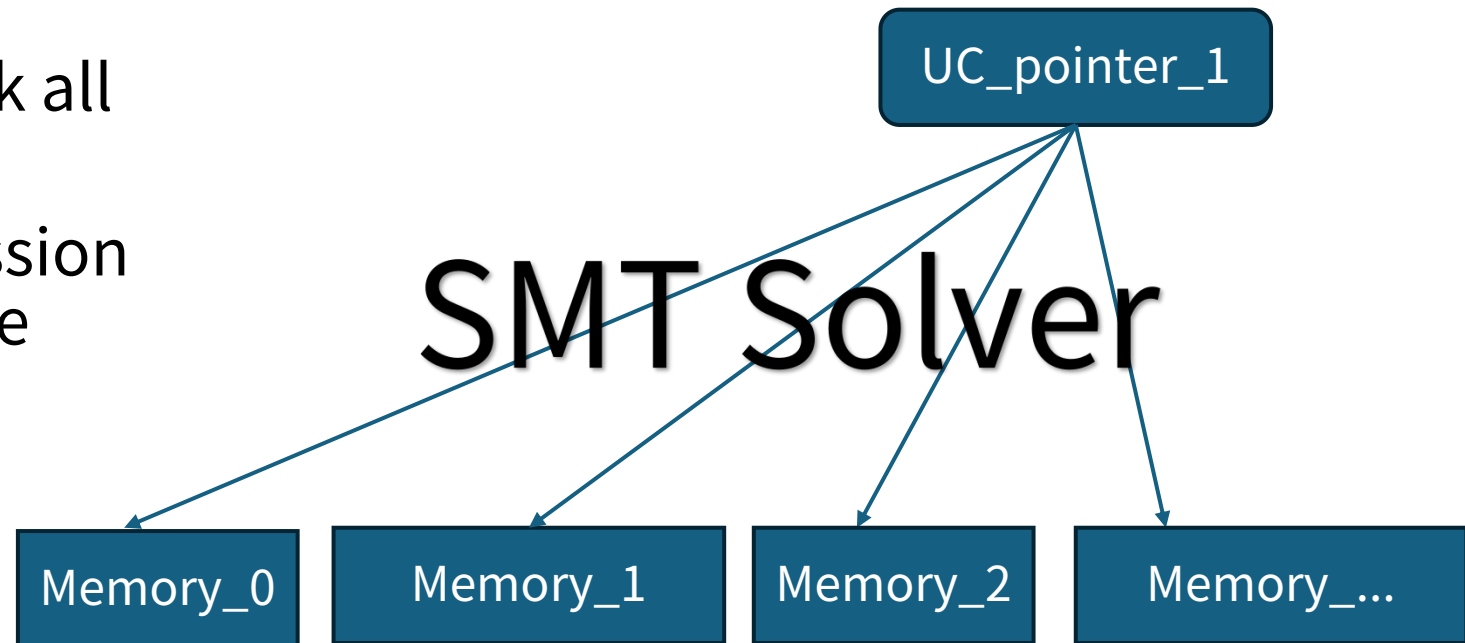
One **limitation** of existing UC Symbolic Execution



Where?

Symbolic Pointer Reasoning in Under-Constrained Symbolic Execution

- Current solution in KLEE
- Use the SMT solver to check all memory object one by one
- Unless the symbolic expression has a unique concrete value



Motivating Example

```
1. struct A {int a1; int a2;};
2.
3. struct B {struct A b1; int b2;};
4.
5. void handler(long arg1, long arg2, long user_inputs) {
6.   struct A* var1= (struct A*)arg1;           // bitcast sym1
7.   if (var1->a2 == 1) {                         // load sym1+4
8.     struct B* var2 = container_of(var1, struct B, b1); // bitcast sym1
9.     var2->b2 = 1;                             // store sym1+8
10.  }
11.  struct A* var3= (struct A*)arg2;           // bitcast sym2
12.  var2->a2 = 2;                             // store sym2+4
13.  if (var1->a2 == 2) {                       // load sym1+4
14.    ... // code related to user_inputs
15.  }
16. }
```

- Resolve symbolic pointers when encounter memory operation instructions

MO1:



error

MO2:



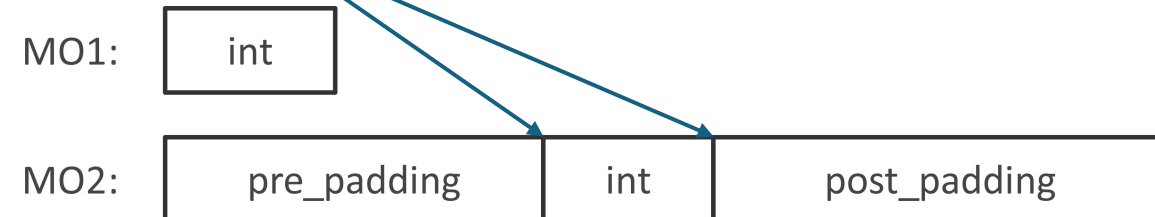
Motivating Example

```
1. struct A {int a1; int a2;};
2.
3. struct B {struct A b1; int b2;};
4.
5. void handler(long arg1, long arg2, long user_inputs) {
6.   struct A* var1= (struct A*)arg1;           // bitcast sym1
7.   if (var1->a2 == 1) {                         // load sym1+4
8.     struct B* var2 = container_of(var1, struct B, b1); // bitcast sym1
9.     var2->b2 = 1;                             // store sym1+8
10.  }
11.  struct A* var3= (struct A*)arg2;           // bitcast sym2
12.  var2->a2 = 2;                             // store sym2+4
13.  if (var1->a2 == 2) {                       // load sym1+4
14.    ... // code related to user_inputs
15.  }
16. }
```

1) May still not be large enough, leading to continued out-of-bound errors

Increases memory usage and requires more time to handle state forking

Places an additional computational burden on the SMT solver

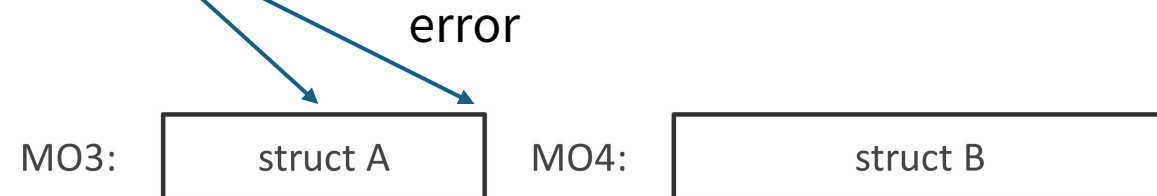


How about alloc a large memory?

Motivating Example

```
1. struct A {int a1; int a2;;}
2.
3. struct B {struct A b1; int b2;;}
4.
5. void handler(long arg1, long arg2, long user_inputs) {
6.   struct A* var1= (struct A*)arg1;           // bitcast sym1
7.   if (var1->a2 == 1) {                         // load sym1+4
8.     struct B* var2 = container_of(var1, struct B, b1); // bitcast sym1
9.     var2->b2 = 1;                             // store sym1+8
10.  }
11.  struct A* var3= (struct A*)arg2;           // bitcast sym2
12.  var2->a2 = 2;                             // store sym2+4
13.  if (var1->a2 == 2) {                       // load sym1+4
14.    ... // code related to user_inputs
15.  }
16. }
```

- Resolve symbolic pointers when encounter types of pointers



Motivating Example

```
1. struct A {int a1; int a2;;}
2.
3. struct B {struct A b1; int b2;;}
4.
5. void handler(long arg1, long arg2, long user_inputs) {
6.   struct A* var1= (struct A*)arg1;           // bitcast sym1
7.   if (var1->a2 == 1) {                         // load sym1+4
8.     struct B* var2 = container_of(var1, struct B, b1); // bitcast sym1
9.     var2->b2 = 1;                             // store sym1+8
10.  }
11.  struct A* var3= (struct A*)arg2;           // bitcast sym2
12.  var2->a2 = 2;                             // store sym2+4
13.  if (var1->a2 == 2) {                       // load sym1+4
14.    ... // code related to user_inputs
15.  }
16. }
```

- Resolve symbolic pointers when encounter types of pointers

MO3:

struct A

MO4:

struct B

Difficult to accurately determine the correct size of the memory object

Motivating Example

```
1. struct A {int a1; int a2;};
2.
3. struct B {struct A b1; int b2;};
4.
5. void handler(long arg1, long arg2, long user_inputs) {
6.   struct A* var1= (struct A*)arg1;           // bitcast sym1
7.   if (var1->a2 == 1) {                         // load sym1+4
8.     struct B* var2 = container_of(var1, struct B, b1); // bitcast sym1
9.     var2->b2 = 1;                             // store sym1+8
10.  }
11.  struct A* var3= (struct A*)arg2;           // bitcast sym2
12.  var2->a2 = 2;                             // store sym2+4
13.  if (var1->a2 == 2) {                       // load sym1+4
14.    ... // code related to user_inputs
15.  }
16. }
```

- Directly create new memory objects for sym2?
- How about `sym2 == sym1`?
- Otherwise, the specifications of the user inputs will be lost.

?

MO3:

struct A

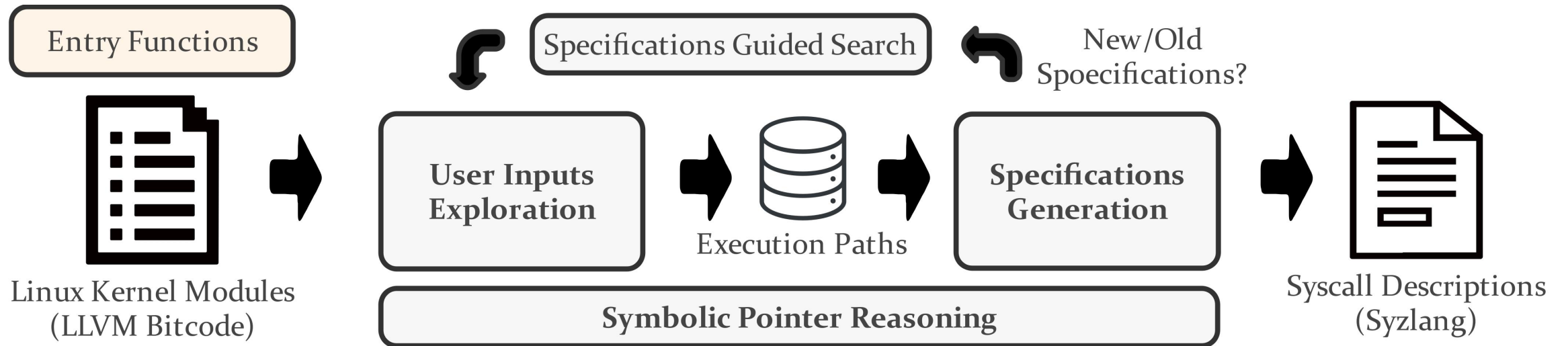
MO4:

struct B



Workflow

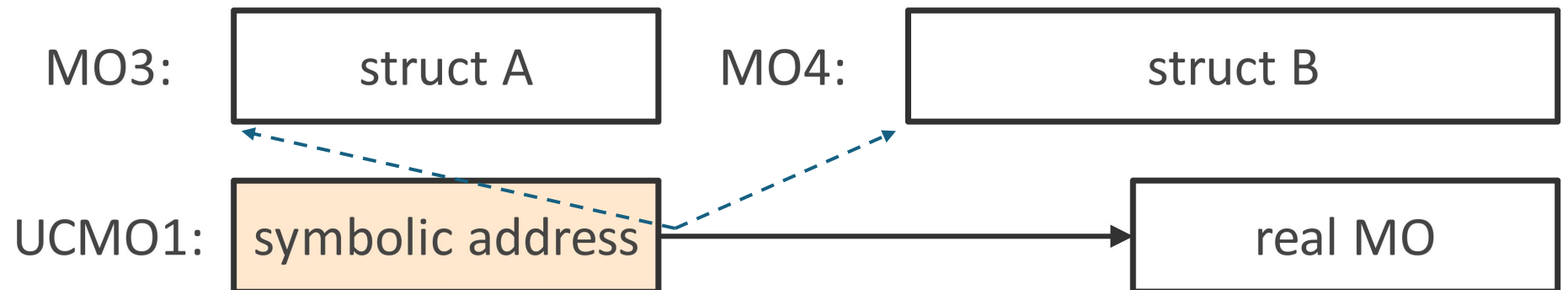
8.6k (C++) lines of code
to adapt KLEE for
SyzSpec



Symbolic Pointer Reasoning

Under-Constrained Memory Object (UCMO)

- Dynamic relocation and resizing of memory objects during symbolic execution



Symbolic Pointer Reasoning

Resolves the symbolic pointer base address + offset

Cheaper Type-based Analysis + Expensive Constraint Solving

Base address An existing UCMO? New UCMO?

Relocate and Resize the UCMO if needed

Symbolic Pointer Reasoning

Symbolic Pointer (Symbolic) Base address + offset

Cheaper Type-based
Analysis

All UCMOs

Candidate UCMOs

look up the recorded info
to recursively find the
possible base addresses

Symbolic Pointer Reasoning

Symbolic Pointer (Symbolic) Base address + offset

Cheaper Type-based
Analysis

All UCMOs

Candidate UCMOs

Expensive Constraint
Solving

Path Constraints &
Memory
Constraints

Possible UCMOs

UCMO1:

symbolic address

real MO

Memory Constraints:

- 1) symbolic base address of UCMO = concrete address of its corresponding real memory objects
- 2) Symbolic address $\leq$ Base address && Base address $\leq$ Symbolic address + size

Base
address

UCMO1:

symbolic address

Symbolic Pointer Reasoning

Symbolic Pointer (Symbolic) Base address + offset

Cheaper Type-based
Analysis

All UCMOs

Candidate UCMOs

Expensive Constraint
Solving

Path Constraints &
Memory
Constraints

Possible UCMOs

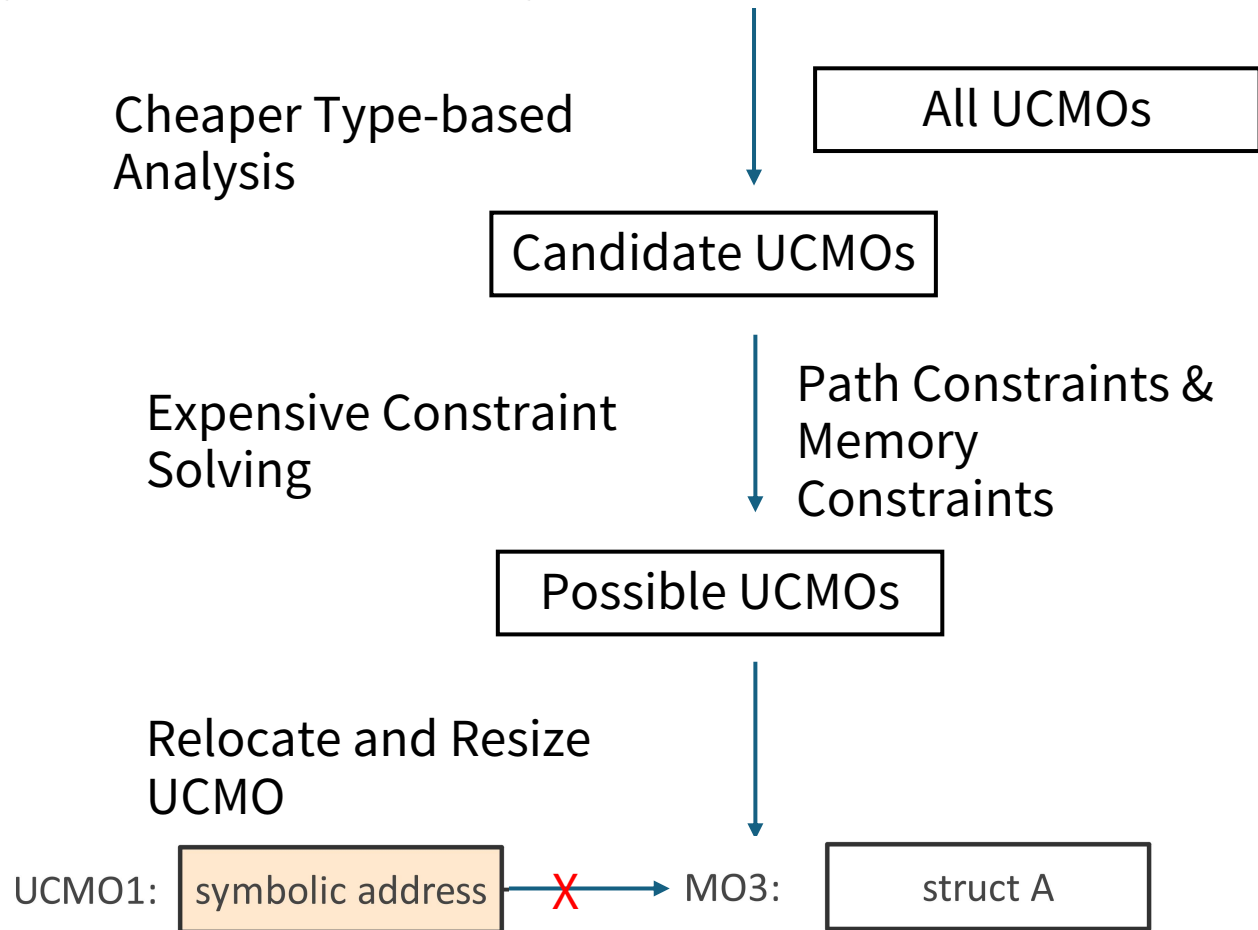
Relocate and Resize
UCMO

UCMO1: symbolic address

X

MO3:

struct A



Symbolic Pointer Reasoning

Symbolic Pointer (Symbolic) Base address + offset

Cheaper Type-based
Analysis

All UCMOs

Candidate UCMOs

Expensive Constraint
Solving

Path Constraints &
Memory
Constraints

Possible UCMOs

Relocate and Resize
UCMO

UCMO1:

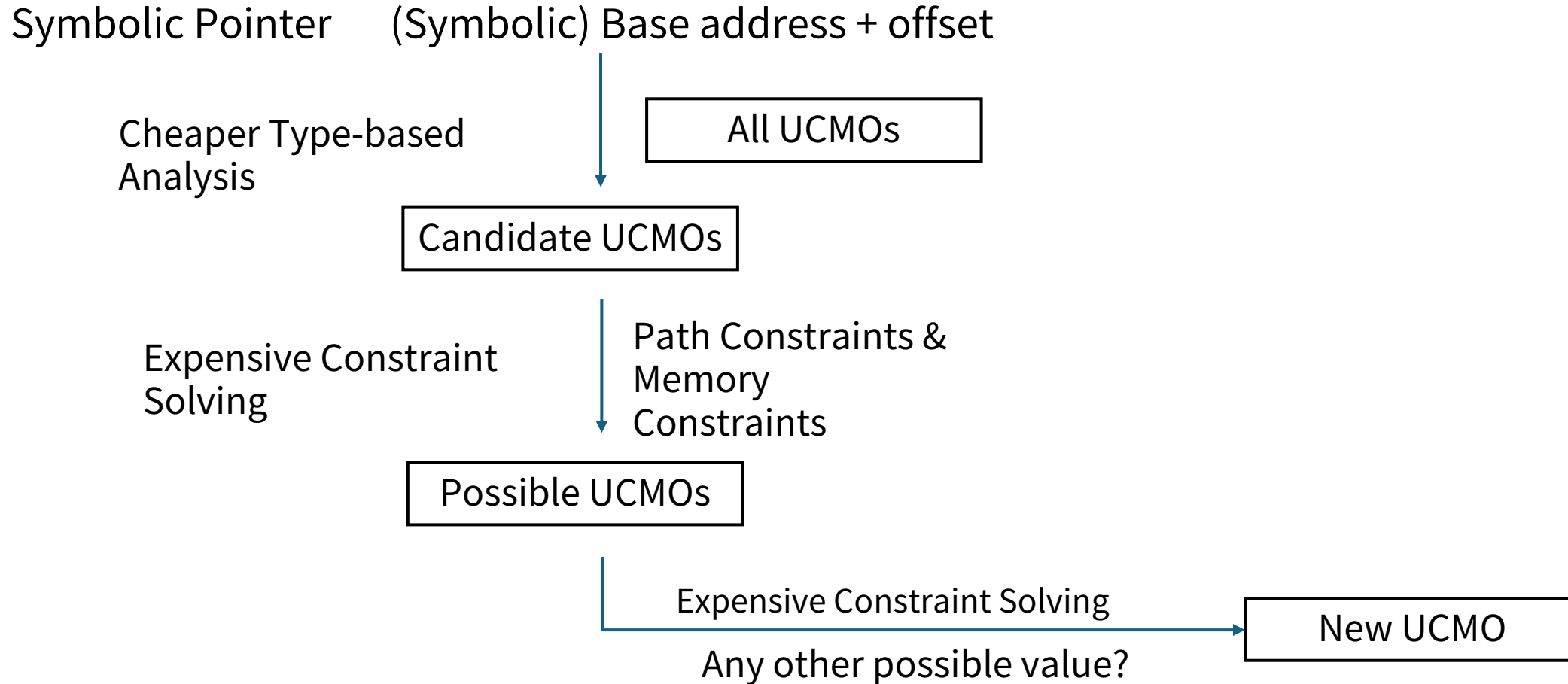
symbolic address

MO4:

struct B



Symbolic Pointer Reasoning



Function Pointer

A function pointer should reference an existing function

- A static type-based method analysis
- Symbolic pointer reasoning

Symbolic Function Pointer Reasoning

Address-taken
functions



Static type
compatible?



Constraints
satisfied?

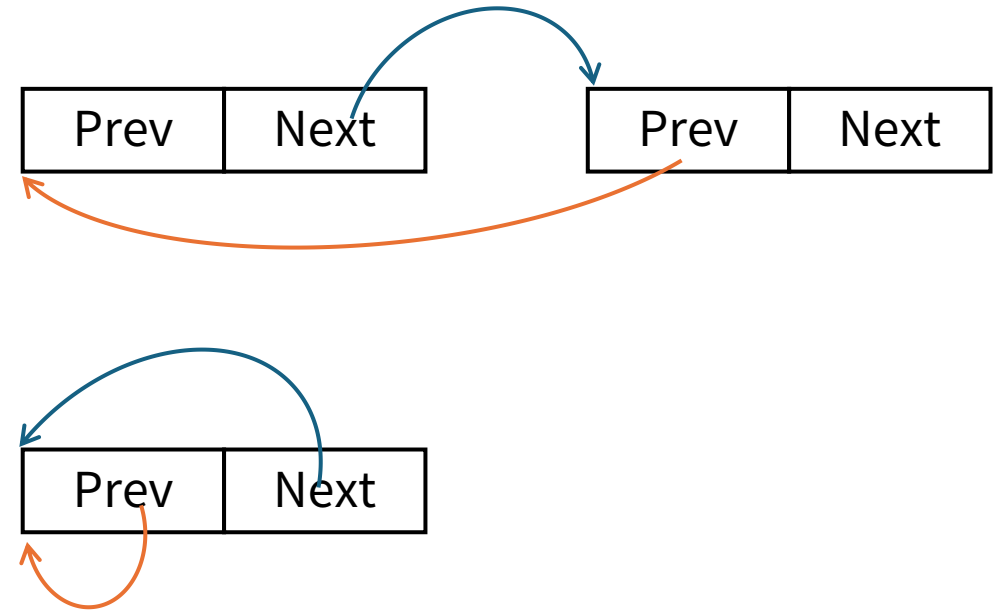


Possible callee
targets

Existing static pointer analysis

Linked List

- Symbolic Pointer Reasoning
 - $\text{list} \rightarrow \text{next} = \text{new list}$
 - Or $\text{list} \rightarrow \text{next} = \text{list}$
-
- Additional Constraints
 - $\text{list} \rightarrow \text{next} \rightarrow \text{prev} = \text{list}$
 - $\text{list} \rightarrow \text{prev} \rightarrow \text{next} = \text{list}$



Specification Generation

- Basic supported types of user inputs in syzlang

Primitive types
(e.g., integers with
different widths
and char)

Pointers

Arrays

Structures
(including unions)

Primitive Types

Symbolic Expressions of Primitive Types

Symbolic Pointer Reasoning

- The integer is actually a pointer

SMT Solver

- Possible values or range

Pointers

Recover the info of the pointer from the associated UCMO

Direction



- Input: Has been read
- Output: Has been written
- Input/Output: Both

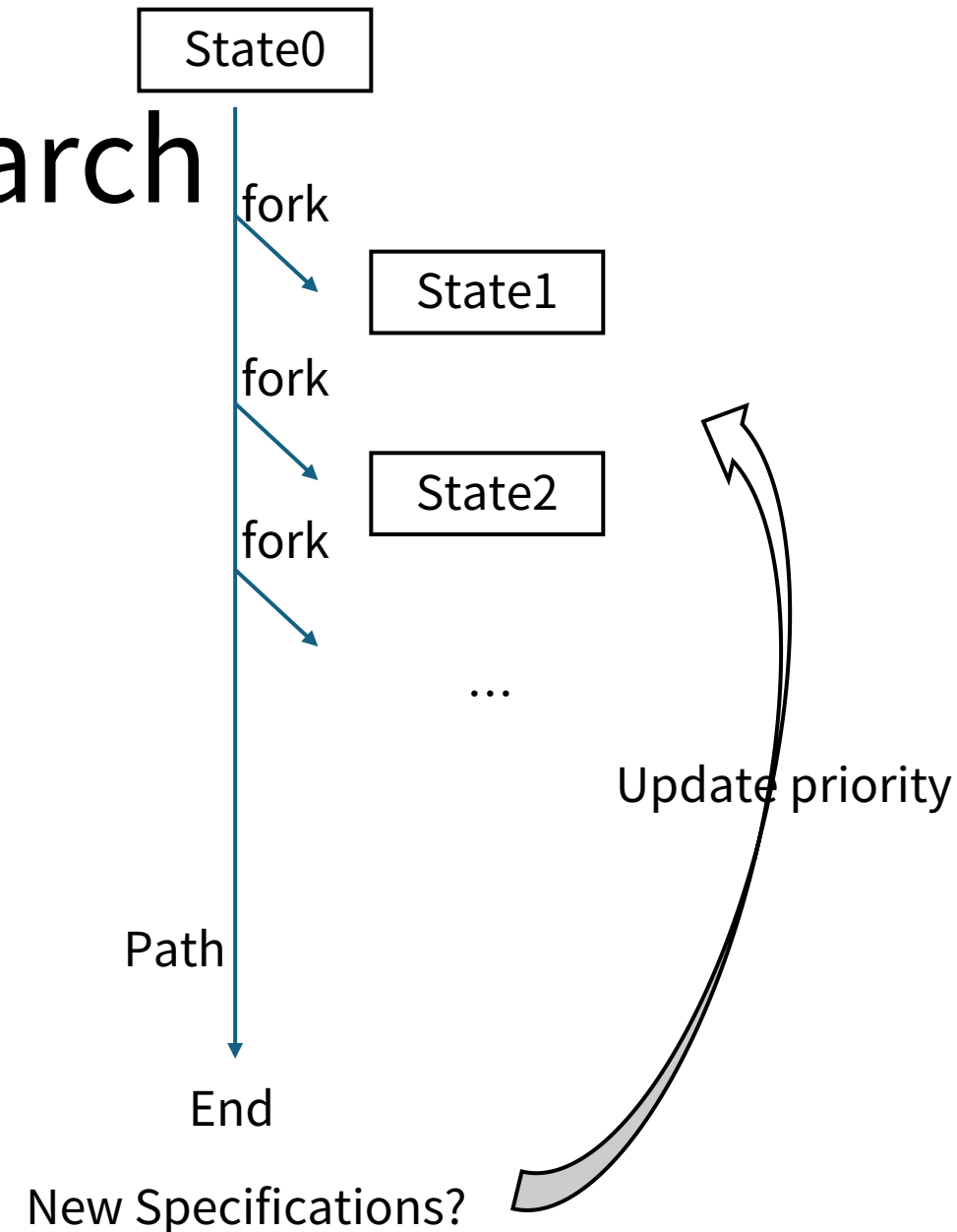
Type



- Size: Prefer larger size, more valuable for fuzzing
- Type: Struct > Union > Array

Specifications Guided Search

- Inspired by coverage-guided fuzzing
- Prioritize the exploration based on whether a path contributed to new specifications



Evaluation

- Dataset
 - 10 drivers in different categories
 - one non-driver kernel module
 - io_uring

Name	Category	Syscalls
capi20	Communication	ioctl, read, write
i2c	Communication	ioctl, read, write
infiniband	Networking	read, write
input_event	Input Devices	ioctl, read, write
mapper_control	Device Management	ioctl
ppp	Networking	ioctl, read, write
ptmx	Pseudo Terminals	ioctl
sg	SCSI Generic	ioctl, read, write
snd_seq	Sound	ioctl, read, write
uinput	Input Devices	ioctl, read, write
iouring	I/O Optimization	io_uring_register

Effectiveness of SyzSpec in Fuzzing

- Compare with SyzDescribe, KSG, SyzGen++, and manually written syscall descriptions from the official syzkaller project
- 24 hours fuzzing with 4 CPU cores (2 QEMU * 2 cores)
- Three separate runs per driver
- Linux kernel v6.6 (latest LTS, 10/29/2023)

Effectiveness of SyzSpec in Fuzzing

Name	Syzkaller	SyzSpec	SyzDescribe	KSG	SyzGen++
capi20	2,881.7	3,189.3	2,756.3	2,996.7	N/A
i2c	4,681.3	6,167.3	4,698.0	N/A	N/A
infiniband	7,073.7	4,708.3	N/A	N/A	N/A
input_event	4,749.7	6,382.3	5,286.3	N/A	N/A
mapper	N/A	3,223.0	1,997.7	N/A	N/A
ppp	6,253.7	6,124.3	5,887.3	N/A	5,633.0
ptmx	14,190.3	10,099.0	10,213.7	10,232.3	10,879.0
sg	7,023.0	9,813.7	8,352.3	N/A	7,528.7
snd_seq	4,384.3	4,835.0	N/A	2,606.3	4,878.7
uinput	5,759.0	4,493.7	4,531.0	N/A	N/A
iouring	6,716.0	6,656.7	N/A	N/A	N/A

Table 2: Comparison of the average code coverage between SyzSpec and previous work.

SyzSpec vs. SyzDescribe

- SyzDescribe can generate syscall descriptions for all 10 drivers, and it explicitly indicates that 2 of these descriptions are incomplete.
- SyzDescribe cannot recover complex user inputs.

Name	SyzSpec	SyzDescribe
capi20	3,189.3	2,756.3
i2c	6,167.3	4,698.0
infiniband	4,708.3	N/A
input_event	6,382.3	5,286.3
mapper	3,223.0	1,997.7
ppp	6,124.3	5,887.3
ptmx	10,099.0	10,213.7
sg	9,813.7	8,352.3
snd_seq	4,835.0	N/A
uinput	4,493.7	4,531.0
iouring	6,656.7	N/A

SyzSpec vs. SyzGen++

- SyzGen++ successfully generates syscall descriptions for 4 drivers in our dataset.
- There are conditions whose values come from global memory, as well as symbolic pointers and function pointers.

Name	SyzSpec	SyzGen++
capi20	3,189.3	N/A
i2c	6,167.3	N/A
infiniband	4,708.3	N/A
input_event	6,382.3	N/A
mapper	3,223.0	N/A
ppp	6,124.3	5,633.0
ptmx	10,099.0	10,879.0
sg	9,813.7	7,528.7
snd_seq	4,835.0	4,878.7
uinput	4,493.7	N/A
iouring	6,656.7	N/A

Performance of SyzSpec

Name	Time	Total Path	Partially Completed Path	Partially Completed / Total
capi20	2.29	209	119	56.94%
i2c	1.86	12,628	8,403	66.54%
infiniband	0.13	567	136	23.99%
input_event	4.49	17,451	12,532	71.81%
mapper_control	0.27	627	359	57.26%
ppp	10.75	4,257	2,410	56.61%
ptmx	16.60	16,934	13,832	81.68%
sg	0.54	5,154	3,450	66.94%
snd_seq	7.01	5,197	3,211	61.79%
uinput	8.00	422	100	23.70%
iouring	7.31	23,297	19,711	84.61%

Table 4: Execution time required by SyzSpec to generate syscall descriptions for those 11 kernel modules.

- The execution times range from 0.13 hours to 16.60 hours, depending on the module.
- The number of execution paths analyzed by SyzSpec ranges from 209 to 23,297.
- There is no direct correlation between the execution time and the number of paths analyzed. Some paths take longer to analyze because they involve more complex constraint-solving processes.

Bug Finding Capability of SyzSpec

Risk	Sum	C repro	Syzlang repro	no repro
out-of-bounds	1			1
use-after-free	4	1		3
use-before-initialization	6	1		5
null-ptr-deref	29	7	3	19
corrupted stack	4	2	1	1
corrupted list	9	3		6
corrupted lock	2	2		0
page fault	4	1		3
deadlock	5	2	1	2
task hung	12	2		10
logic bug	10	2	1	7
Sum	86	23	6	57

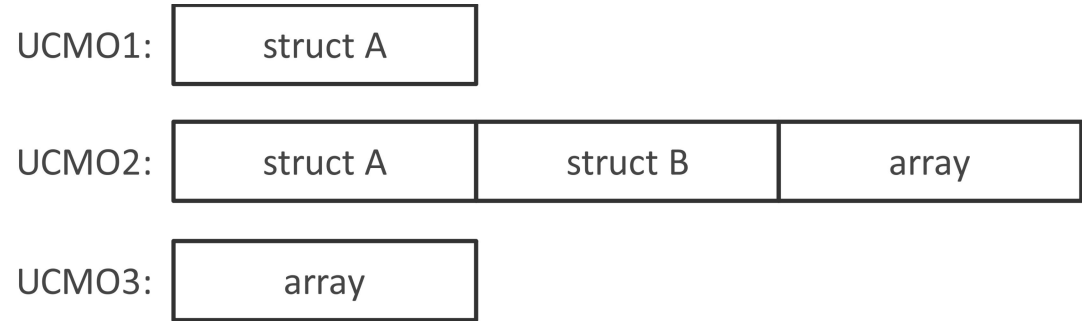
Table 5: Overview of the unknown crashes.

- One-month fuzzing campaign
- On the latest stable version of the Linux kernel (v6.10, released on 07/14/2024)
- Using 70 CPU cores
- After cross-validation with bugs in syzbot, we find that 86 previously unknown crashes, in 11 different risk types.

Thanks!

- our open-source repo:
- <https://github.com/seclab-ucr/SyzSpec>

Type of UCMO



UCMO1

- Only a single type and its size matches the size of the entire object
- Directly infer the type

UCMO2

- Multiple segments (at different offsets), each with its own type
- Construct a new struct type

UCMO3

- No type information is available
- An array of bytes whose size matches the UCMO

Arrays

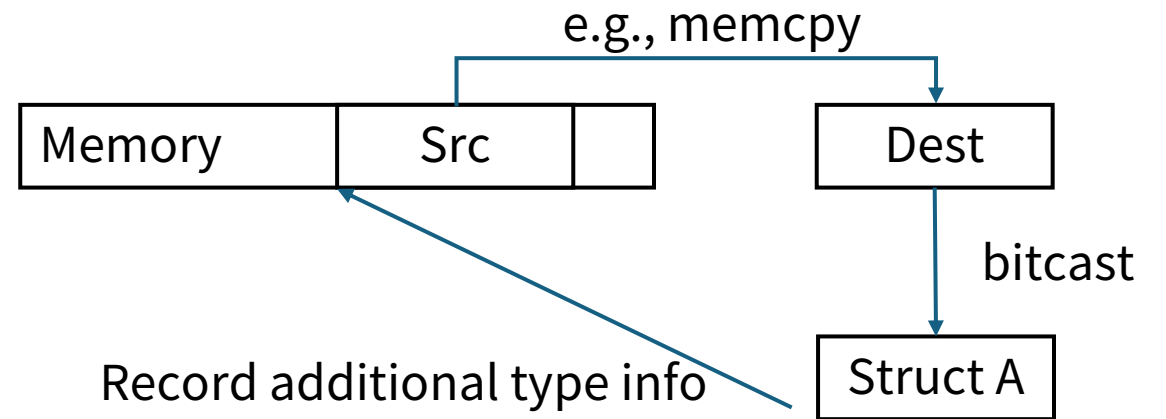
- Only recover their overall size
- Without retrieving the constraints for individual elements

Structures

- Recursively retrieve the type information and constraints for each field

Additional Type Information Collection

- Memory copy operations
 - copy_from_user
 - copy_to_user
 - memdup
 - memcpy



Assembly Code

- Certain assembly code blocks involving memory operations
 - put_user
 - get_user
- Return under-constrained symbolic expressions for assembly code with return values

Specification Merge

No new specifications

- Discard the current specifications to avoid redundancy

New specifications

- Server as feedback for the specifications guided search
- Merge them with existing specifications or keep them separate

SyzSpec vs. SyzDescribe

- SyzDescribe can generate syscall descriptions for all 10 drivers, and it explicitly indicates that 2 of these descriptions are incomplete.
- SyzDescribe cannot recover complex user inputs.

Name	SyzSpec	SyzDescribe
capi20	3,189.3	2,756.3
i2c	6,167.3	4,698.0
infiniband	4,708.3	N/A
input_event	6,382.3	5,286.3
mapper	3,223.0	1,997.7
ppp	6,124.3	5,887.3
ptmx	10,099.0	10,213.7
sg	9,813.7	8,352.3
snd_seq	4,835.0	N/A
uinput	4,493.7	4,531.0
iouring	6,656.7	N/A

SyzSpec vs. KSG

- KSG can generate syscall descriptions for all 10 drivers. But for 7 of them, it cannot include any type information or value constraints.
- KSG only performs an intra-procedural analysis.
- For the `snd_seq` module, KSG cannot handle symbolic pointers effectively.

Name	SyzSpec	KSG
capi20	3,189.3	2,996.7
i2c	6,167.3	N/A
infiniband	4,708.3	N/A
input_event	6,382.3	N/A
mapper	3,223.0	N/A
ppp	6,124.3	N/A
ptmx	10,099.0	10,232.3
sg	9,813.7	N/A
snd_seq	4,835.0	2,606.3
uinput	4,493.7	N/A
iouring	6,656.7	N/A

SyzSpec vs. SyzGen++

- SyzGen++ successfully generates syscall descriptions for 4 drivers in our dataset.
- There are conditions whose values come from global memory, as well as symbolic pointers and function pointers.

Name	SyzSpec	SyzGen++
capi20	3,189.3	N/A
i2c	6,167.3	N/A
infiniband	4,708.3	N/A
input_event	6,382.3	N/A
mapper	3,223.0	N/A
ppp	6,124.3	5,633.0
ptmx	10,099.0	10,879.0
sg	9,813.7	7,528.7
snd_seq	4,835.0	4,878.7
uinput	4,493.7	N/A
iouring	6,656.7	N/A

SyzSpec vs. manually written syscall descriptions

- The official syzkaller provides syscall descriptions for 9 drivers and the io_uring module.
- SyzSpec achieves better coverage in 6 kernel modules, while the official syscall descriptions perform better in 3 modules.
- SyzSpec is the only automated tool that supports the non- driver io_uring module.

Name	SyzSpec	Syzkaller
capi20	3,189.3	2,881.7
i2c	6,167.3	4,681.3
infiniband	4,708.3	7,073.7
input_event	6,382.3	4,749.7
mapper	3,223.0	N/A
ppp	6,124.3	6,253.7
ptmx	10,099.0	14,190.3
sg	9,813.7	7,023.0
snd_seq	4,835.0	4,384.3
uinput	4,493.7	5,759.0
iouring	6,656.7	6,716.0

Effectiveness of SyzSpec in Fuzzing

Name	Syzkaller	SyzSpec	SyzDescribe	KSG	SyzGen++
capi20	0.0	0.0	0.0	0.0	N/A
i2c	0.0	0.3	0.0	N/A	N/A
infiniband	0.0	0.7	N/A	N/A	N/A
input_event	0.0	0.0	0.0	N/A	N/A
mapper	N/A	0.7	0.0	N/A	N/A
ppp	0.0	0.3	0.0	N/A	0.3
ptmx	1.3	0.0	0.0	0.0	0.0
sg	3.3	2.7	1.7	N/A	2.0
snd_seq	1.3	0.0	N/A	0.3	0.0
uinput	1.7	1.0	0.7	N/A	N/A
iouring	0.0	0.0	N/A	N/A	N/A

Table 3: Comparison of the average number of unique crashes between SyzSpec and previous work.

Discussion and Limitations

- Specifications Merge
 - Currently, we only merge specifications of the user inputs with the same type.
 - However, there are situations where specifications of different types can also be merged.
 - These potential merges heavily rely on heuristics, and it is challenging to encode all of them.
 - LLMs are possible solution.

```
1. struct A {int a_1; int a_2;}  
2. struct B {struct A b_1; struct A b_2;}  
3. struct C {struct A c_1; struct A c_2; struct A c_3}  
4. array[struct A]
```

Discussion and Limitations

- Entry Functions
 - Currently, the SyzSpec requires entry functions as input.
 - However, existing tools are limited to automatically recovering entry functions specifically for drivers with hard-coded domain knowledge.
 - It would be useful to develop a more generalized method to recover handler functions for syscall interfaces.
 - It is possible to leverage large language models, which already have significant knowledge about the Linux kernel and syscall internals, to automatically extract the necessary domain knowledge.